# Unity Design System Component Consolidation Upgrade Guide

2025-02-06

## Purpose

The Unity Design System has been refactored to improve usability and sustainability, add functionality, upgrade libraries, and modernize. This document details what you should know about these changes and how to apply them to your products and projects that use Unity.

## Who is this for

This document provides details for those consuming Unity components in their products and projects to enable them to upgrade with minimal disruption. Those who use products such as Webspark 2, which implement Unity, should be minimally impacted. The work to update belongs with the product development team rather than product users.

## What has changed

- Package name realignment
  - `components-core` is deprecated and consumers should now use `unity-react-core`
  - `component-carousel` is now in `unity-react-core`

    - `component-header` and `component-footer` have been combined into `component-header-footer` (a standalone component with no dependency on the `unity-bootstrap-theme`)
- Version Updates
    - React version is upgraded from 16 to 18
    - Yarn is upgraded from version 1 to 4
- Storybook Updates
    - Location of theme stories used by Storybook is now `unity-react-core`. When visiting https://unity.web.asu.edu, reference stories for theme components will now be in that package.
    - Storybook stories now include Bootstrap and React references and both can be viewed within a single story using the Storybook controls.
- The Unity Design System has migrated from Webpack to Vite for packaging and transpilation.
- In `unity-react-core`, the `cardTitle` prop on the `Button`, `ButtonIconOnly` and `ButtonTag` components is now deprecated and will be removed in the future.

# Changes to built packages and output files

Notes below reflect change in purpose, scope or interface. They do not attempt to document internal changes to imports and code within components. Care should be taken to ensure you use versions of components compatible with their dependencies. See the **Version compatibility** section for more detail on version compatibility.

- `app-degree-pages`
    - anonymous profile image location is now in dist/assets/anon.png. Previously was at `dist/img/anon.png`.
- `app-rfi`
    - No changes
- `component-carousel`
    - Deprecated and removed.
    - Now found in `unity-react-core`
- `component-events`
    - `vendor.umd.js` is no longer packaged separately. It is all one file under the same `asuEvents.umd.js`

- `component-footer`
    - Deprecated but not removed from repo for short term. Consolidated into one package `component-header-footer`.
    - Removed vendor file in deprecated package and dist file is same name, `asuFooter.umd.js`
- `component-header`
    - Same as footer, above.
- `component-header-footer`
    - New package that includes both header and footer.
    - dist file is named `asuHeaderFooter.[format].js` and vendor file is now bundled with it instead of into a separate file.
    - Package now longer requires Font Awesome as a dependency.
    - init functions are the same as previous.
    - The components in this package can be used independent of the `unity-bootstrap-theme` package.
- `component-news`
    - Same as component-events
- `components-core`
    - Deprecated but kept in repo for any potential immediate updates needed.
    - Now under the unity-react-core package
- `unity-react-core`
    - Replaces components-core moving forward.
    - Main dist bundles found in `dist/unityReactCore.[format].js`
    - ES module components are also bundled individually in the `dist/esm/components/${componentName}.es.j`s. For example the ranking card will be `dist/esm/components/RankingCard.es.js`.
        - These are only for use in environments where ES modules are supported. If you are using a browser to initialize these, use the `dist/unityReactCore.umd.js` file and use the init functions provided.

- `unity-bootstrap-theme`
    - No change. Redundant reference stories for Storybook will remain in this package in the short term to enable smooth transition, but eventually these will be removed in favor of the stories in `unity-react-core`.

# New features

- `unity-react-core` now includes React versions of most components from the `unity-bootstrap-theme`. Storybook UI now includes React and Bootstrap details.
- Future new components will automatically be included in `unity-react-core`. Flexibility to implement as either HTML or as React.
- https://unity.web.asu.edu site is now a React application and can be used as a reference for how to consume and implement Unity in a React application. See https://github.com/ASU/asu-unity-stack/tree/dev/packages/static-site to view the code.
- Added `GaEventWrapper` helper to enable consistent data layer implementation pattern in components. React components handle the data layer events internally and Storybook instances of components won't include `data-ga-*` attributes in their markup.
- Unity has begun to use TypeScript internally in places. This won't require any changes on the part of applications using Unity React components, but may provide an enhanced developer experience.

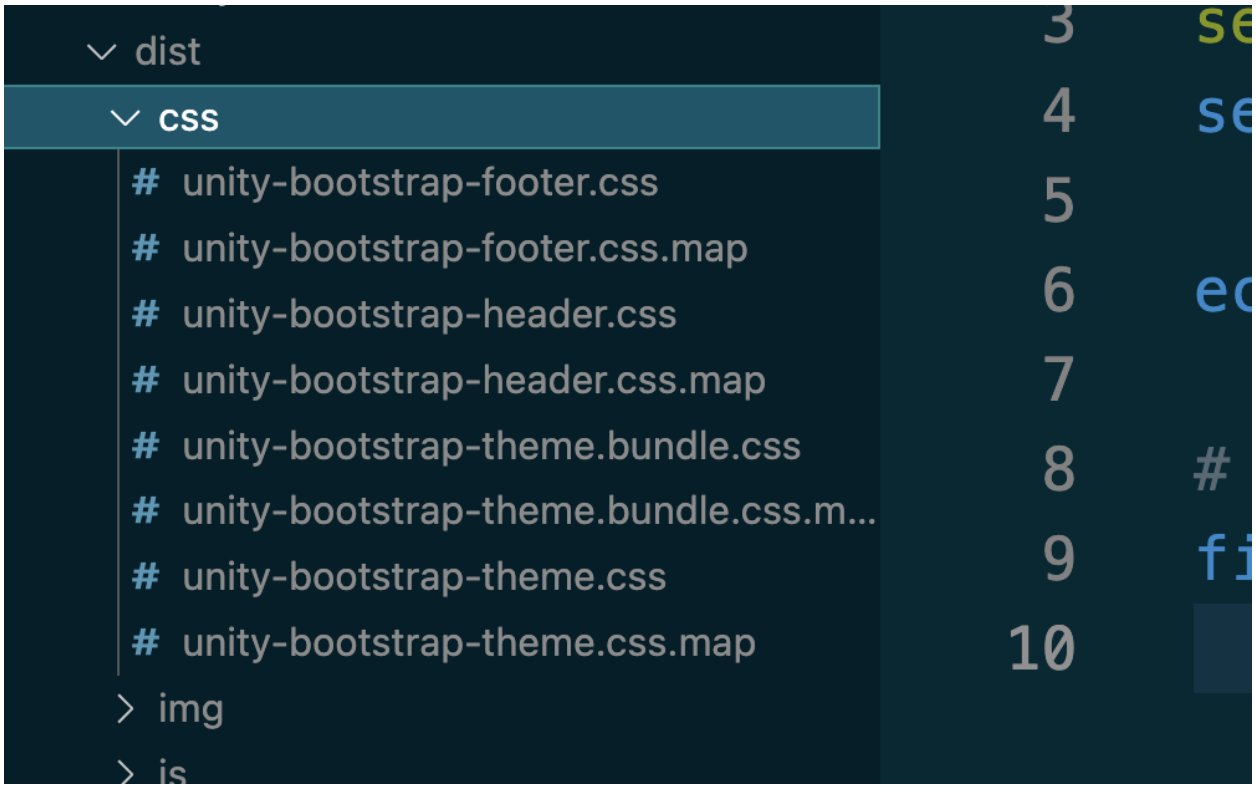
# How to upgrade

- Take note of the new package names and update your imports accordingly. Init functions and props stay the same.
- The footer now in `component-header-footer` no longer requires the `unity-bootstrap-theme` package.
- Applications using the `unity-bootstrap-theme` should be unaffected by this change, and no special steps to upgrade are needed.
- Stories in the `unity-bootstrap-theme` have been relocated to the unity-react-core package and will be maintained and updated there. The styles used by these stories will continue to be maintained in and provided by the `unity-bootstrap-theme` package.

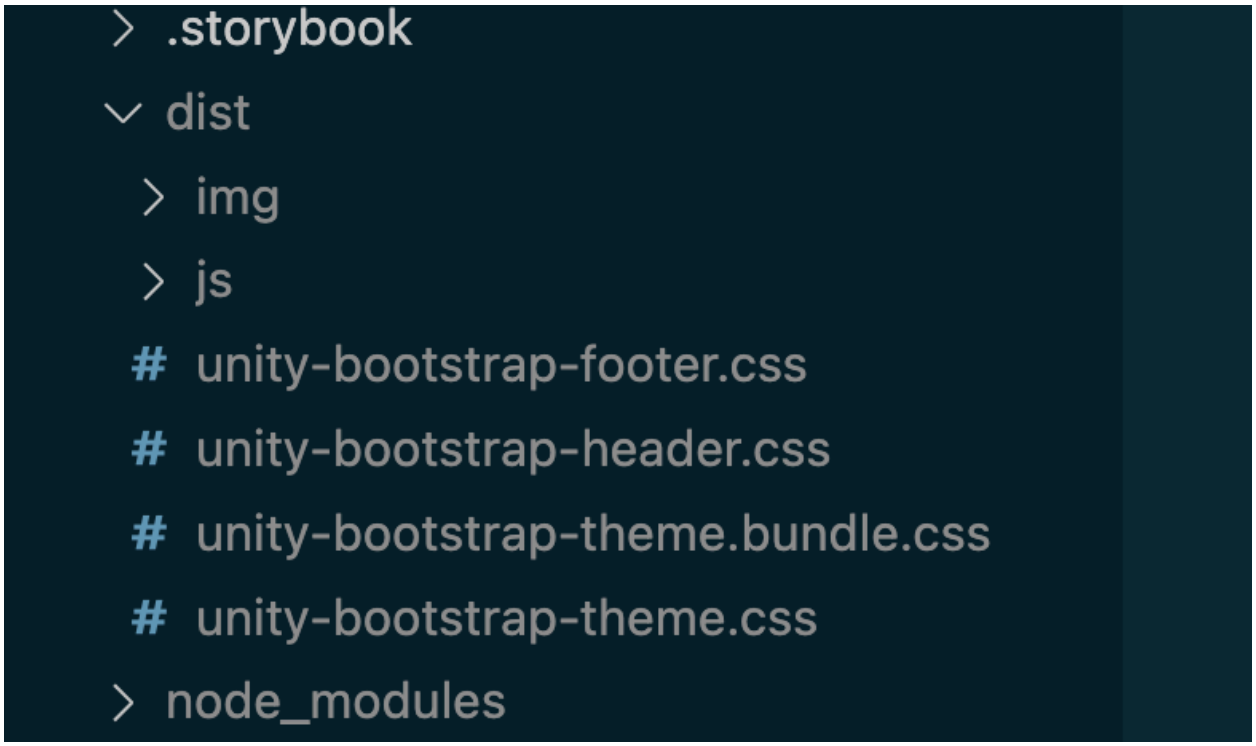
# Using unity-bootstrap-theme

- `unity-bootstrap-theme` can now be imported in React applications: `import '@asu/unity-bootstrap-theme';`
- Unity consumers that compile their own packages to create CMS themes, may need to rework their routines.
    - Paths have changed for CSS assets in `unity-bootstrap-theme`.

## Old CSS asset locations




## New CSS asset locations



# Version compatibility

When updating to the new post-consolidation packages, it is recommended to upgrade all packages at that time.

| Package | Final pre-consolidation version | First post-consolidation version |
|---|---|---|
| `app-degree-pages` | 2.6.2 | 3.0.0 |
| `app-rfi` | 2.4.2 | 3.0.0 |
| `app-webdir-ui` | 4.1.2 | 5.0.0 |
| `component-carousel` | 2.4.0 | Removed. Use unity-react-core |
| `component-cookie-consent` | 2.1.0 | No change. |
| `component-events` | 2.1.2 | 3.0.0 |
| `component-footer` | 2.2.3 | Deprecated. No longer updated. Use component-header-footer |
| `component-header-footer` | NA | 1.0.1 |
| `component-header` | 2.3.3 | Deprecated. No longer updated. Use component-header-footer |
| `component-news` | 3.1.0 | 4.0.0 |
| `components-core` | 4.1.1 | Deprecated. Use unity-react-core |
| `unity-bootstrap-theme` | 1.20.2 | 1.21.0 |
| `unity-react-core` | NA | 1.0.1 |

# FAQ

## I'm not ready to upgrade yet. How does this impact me?

Historical packages will remain available. You can upgrade on your own schedule. Package versions in most cases involve a major breaking change update, so applications that automatically pull in the latest minor update shouldn't be negatively impacted. Packages that have been deprecated will no longer receive updates but will remain usable as long as their dependencies are met with compatible versions of other Unity packages.